

软件开发计划书v2

软件开发计划书

文档标识：SDP-LIFE-ASSIST-2025-V1.0

版本号：V1.0

日期：2026年

[TOC]

1 引言

1.1 标识

本文档标识号为 SDP-LIFE-ASSIST-2025-V1.0，适用于综合生活助手平台软件系统的开发工作。文档标题为《综合生活助手平台软件开发计划书》，缩略词为 SDP（Software Development Plan），当前版本号为 V1.0，初始发行号为第1版。

1.2 系统概述

综合生活助手平台（以下简称“本平台”）是一款面向城市居民的本地生活服务类移动与Web综合应用系统，旨在整合外卖订餐、到店消费、休闲娱乐和生活服务等多类业务，实现商家搜索与推荐、外卖点餐、到店团购、用户评价、在线下单与支付等核心功能，致力于连接用户与本地商家，构建“吃、喝、玩、乐、购”一体化生活服务入口。

本系统参考对标平台包括美团（<https://www.meituan.com/>）、饿了么（<https://www.ele.me/>）及大众点评（<https://www.dianping.com/>），属于全新开发项目，暂无历史运行及维护记录。

- 投资方/开发方：本项目小组（孙伟宸、黄羿、孙铭、钟其宏、戴昊）
- 需方/用户：城市居民及本地商家
- 支持机构：课程指导教师及实验环境提供方
- 计划运行现场：开发及测试环境部署于本地服务器及云端测试环境

相关文档包括：软件需求规格说明书、系统设计说明书、用户手册等。

1.3 文档概述

本文档为综合生活助手平台的软件开发计划书，用于描述本项目开发工作的整体安排、所采用的方法、各阶段活动、进度计划、人员组织及资源配置，供项目成员和课程指导教师参考与监督。本文档不涉及保密性或私密性要求，可在项目组内部及指导教师范围内公开共享。

1.4 与其他计划之间的关系

本软件开发计划是项目的核心文档，与以下计划存在关联关系：

- 软件测试计划：依据本计划中的开发阶段安排制定，测试活动与开发活动并行推进；
- 软件配置管理计划：依托本计划中确定的开发环境和版本管理策略执行；
- 项目进度跟踪计划：以本计划中的里程碑节点为基准进行跟踪与更新。

1.5 基线

编写本开发计划的输入基线为：

- 课程大作业选题说明（选题2：生活助手平台）；
- 对标系统调研报告（美团、饿了么、大众点评功能分析）；
- 课程实验指导文档及软件开发计划书模板（软件开发计划书.md）。

2 引用文件

编号	标题	版本/日期
[1]	软件开发计划书模板	软件开发计划书.md, 课程提供版本
[2]	IEEE Std 1058-1998, 软件项目管理计划标准	1998年
[3]	GB/T 8567-2006, 计算机软件文档编制规范	2006年
[4]	美团平台功能参考, https://www.meituan.com/	访问于2025年
[5]	饿了么平台功能参考, https://www.ele.me/	访问于2025年
[6]	大众点评平台功能参考, https://www.dianping.com/	访问于2025年

3 交付产品

3.1 程序

本项目需开发交付的软件程序如下，均属于新开发性质：

- 综合生活助手平台后端服务程序（RESTful API服务，含用户管理、商家管理、订单管理、支付管理等模块）；
- 综合生活助手平台前端Web应用程序（基于Vue.js或React框架的单页面应用）；
- 综合生活助手平台移动端应用程序（Android/iOS端，或移动H5版本）；
- 数据库初始化脚本及数据迁移程序；
- 系统部署脚本及配置文件。

3.2 文档

随软件产品交付的技术文档包括：

- 软件开发计划书（本文档）；
- 软件需求规格说明书；
- 系统设计说明书（含概要设计与详细设计）；
- 用户手册；
- 安装部署手册；
- 测试报告。

3.3 服务

本项目课程范围内不涉及正式的运维服务交付，但需在验收阶段提供系统演示与讲解服务，配合指导教师完成功能验证。

3.4 非移交产品

以下产品为项目开发过程中使用，不随最终产品移交：

- 开发过程中使用的测试数据及测试脚本；
- 开发环境配置文件及本地调试工具；
- 项目内部会议纪要及过程草稿文档。

3.5 验收标准

- 软件功能符合软件需求规格说明书中规定的全部功能性需求；
- 系统核心功能（商家搜索、外卖点餐、到店团购、用户评价、在线支付）可正常运行，无阻断性缺陷；
- 系统在模拟并发用户场景下响应时间满足基本性能要求；
- 提交的文档齐全，格式规范，内容与实现一致；

- 代码符合本计划5.2.2条规定的编码标准，通过代码审查。

3.6 最后交付期限

依据课程安排，各阶段产品交付节点如下：

- 软件需求规格说明书：需求分析阶段结束时提交；
- 系统设计说明书：设计阶段结束时提交；
- 可运行软件系统及全套文档：课程大作业最终截止日期前提交；
- 系统演示：在课程规定的答辩/验收时间进行。

4 所需工作概述

4.1 系统需求与约束

本项目需开发一套面向城市居民的综合生活助手平台，核心需求包括：

- 用户端功能：用户注册与登录、商家搜索与分类浏览、商品/服务浏览与下单、在线支付、订单状态跟踪、用户评价与评分；
- 商家端功能：商家信息展示、菜品/商品管理、订单接收与处理；
- 平台功能：商家推荐算法、评价真实性管理、配送时效展示。

主要约束包括：

- 系统须在课程规定时间内完成，开发周期受限；
- 项目团队规模为5人，人力资源有限；
- 技术选型须在团队现有技术栈范围内，降低学习成本；
- 不涉及真实支付接口对接，支付功能采用模拟实现。

4.2 文档编制需求与约束

项目需按课程要求编制软件开发计划书、需求规格说明书、系统设计说明书、测试报告及用户手册等文档。文档编制应遵循GB/T 8567-2006规范，格式统一，内容与实际开发保持一致。

4.3 系统生命周期定位

本项目覆盖软件生命周期中从需求分析到编码完成并部署上线的阶段，具体包括：需求分析、系统设计、编码实现、测试验证及部署交付。不涉及合同签订、售后运维等环节。

4.4 开发策略

采用迭代增量式开发策略，按功能模块分阶段实现，优先完成核心主流程（用户注册登录、商家浏览、下单支付），再逐步完善推荐、评价等扩展功能。

4.5 进度与资源约束

项目组共5人，须在课程学期内完成全部开发与文档工作。进度安排以课程节点为基准，需合理分配各成员工作量，避免关键路径延误。

4.6 其他约束

- 安全性：用户密码须加密存储，接口需进行基本的鉴权保护；
 - 数据标准：遵循RESTful API设计规范，接口文档须与实现保持同步；
 - 硬件依赖：系统部署于本地或云端测试服务器，无需专用硬件；
 - 开发与测试相互依赖：各模块开发完成后须通过单元测试方可提交集成。
-

5 实施整个软件开发活动的计划

5.1 软件开发过程

本项目采用迭代增量式开发过程，整体划分为以下阶段：

- 第一阶段（需求分析）：开展用户需求调研，参考对标平台，完成软件需求规格说明书；
- 第二阶段（系统设计）：完成系统概要设计和详细设计，确定技术架构、数据库设计及接口规范；
- 第三阶段（编码实现）：按模块分工并行开发，遵循编码标准，进行代码审查；
- 第四阶段（测试验证）：执行单元测试、集成测试和系统功能测试，修复缺陷；
- 第五阶段（部署交付）：完成系统部署，提交全套文档，准备验收演示。

各阶段活动目标、输入输出及里程碑详见第7章进度表。

5.2 软件开发总体状态

5.2.1 软件开发方法

本项目采用以下开发方法：

- 开发模式：迭代增量式开发，每次迭代聚焦于一组功能模块，迭代周期约为一至两周；
- 需求管理：采用用例图和用户故事描述功能需求，使用需求跟踪矩阵维护需求与实现的对应关系；
- 设计方法：前后端分离架构，后端采用MVC模式，前端采用组件化开发模式；
- 版本管理：使用Git进行源代码版本控制，采用主干开发、特性分支合并策略；
- 缺陷管理：使用GitHub Issues或等效工具记录和跟踪问题；
- 代码审查：关键模块提交前须经至少一名其他成员进行代码审查；
- 自动化工具：使用Postman进行接口测试，使用JUnit/PyTest等框架进行单元测试。

5.2.2 软件产品标准

需求标准：采用结构化用例描述，包含用例名称、参与者、前置条件、基本流程、扩展流程和后置条件。

设计标准：数据库设计须包含实体关系图（ER图）；接口设计须符合RESTful规范，使用OpenAPI（Swagger）格式描述。

编码标准：本项目主要使用Java（后端）、JavaScript/TypeScript（前端）进行开发，编码标准如下：

格式标准：

- 缩进统一使用4个空格（Java）或2个空格（JavaScript/TypeScript），不使用Tab字符；
- 每行代码长度不超过120个字符；
- 类名采用UpperCamelCase（大驼峰）命名，方法名和变量名采用lowerCamelCase（小驼峰）命名，常量采用全大写下划线分隔命名；
- 代码块的左花括号不另起一行，与语句同行。

首部注释标准：每个类文件须包含文件头注释，内容包括：类名、功能描述、作者、创建日期、版本号、修改历史。每个公共方法须包含方法注释，内容包括：方法用途、参数说明、返回值说明、异常说明。

其他注释标准：代码中的复杂逻辑、算法或特殊处理须添加行内注释加以说明；注释语言统一使用中文；注释内容须与代码保持同步更新。

命名约定：

- 包名（Java）：全小写，使用反向域名格式，如 `com.project.lifeassist.service`；
- 数据库表名：全小写，单词间以下划线分隔，如 `user_order`；
- 接口路径：全小写，名词复数形式，如 `/api/v1/merchants`；
- 前端组件名：UpperCamelCase，如 `MerchantCard.vue`。

编程限制：

- 禁止使用魔法数字，所有常量须定义为命名常量或枚举；

- 禁止提交包含System.out.println（Java）或console.log（JavaScript，调试用途除外）的代码至主分支；
- 方法圈复杂度不超过10，超过须进行重构拆分。

测试标准：单元测试覆盖率要求核心业务逻辑不低于60%；测试用例须包含正常流程、边界值和异常流程的验证。

5.2.3 可重用的软件产品

5.2.3.1 吸纳可重用的软件产品

本项目将充分利用成熟的开源框架和第三方库，降低重复开发工作量。评估和吸纳可重用软件产品的方法如下：

- 搜寻范围：Maven Central、npm仓库、GitHub开源项目；
- 评估准则：许可证兼容性（须为MIT、Apache 2.0等宽松开源许可）、社区活跃度、文档完整性、版本稳定性；
- 已选定的可重用软件产品：

组件	用途	版本	优点	限制
Spring Boot	后端应用框架	3.x	成熟稳定，文档完善	启动相对较慢
MyBatis-Plus	数据库ORM	3.x	简化CRUD操作	复杂查询需手写SQL
Vue.js / React	前端框架	3.x / 18.x	组件化开发，生态丰富	学习曲线存在
Element Plus	前端UI组件库	2.x	提供丰富业务组件	样式定制有一定复杂度
MySQL	关系型数据库	8.x	稳定可靠，广泛使用	需合理设计索引
Redis	缓存中间件	7.x	高性能，支持多数据结构	需注意缓存一致性
JWT	用户鉴权	—	无状态认证，易于扩展	Token管理需额外处理

5.2.3.2 开发可重用的软件产品

本项目在开发过程中，对于具有通用性的组件将进行模块化封装，以便复用，包括：

- 通用分页查询组件；
- 统一异常处理与响应封装模块；
- 通用文件上传工具类；
- 前端公共组件库（如评分组件、商家卡片组件、订单状态组件）。

上述可重用组件将在代码仓库中独立目录维护，并编写相应使用说明文档。

5.2.4 处理关键性需求

5.2.4.1 安全性保证

- 用户密码采用BCrypt算法加密存储，禁止明文保存；
- 接口访问采用JWT Token鉴权，敏感接口须验证用户权限；
- 防止SQL注入：使用参数化查询，禁止拼接SQL字符串；
- 防止XSS攻击：对用户输入内容进行转义处理；
- 系统部署时关闭不必要的端口和服务。

5.2.4.2 保密性保证

- 用户个人信息（手机号、地址等）在日志中进行脱敏处理；
- 数据库访问凭据不得硬编码于代码中，须使用环境变量或配置文件管理；
- 项目代码仓库设置为私有，仅项目组成员可访问。

5.2.4.3 私密性协议

本项目为课程实验项目，不涉及真实用户隐私数据，测试数据均为模拟数据。在系统设计和实现过程中，遵循最小化数据收集原则，仅采集业务必要的用户信息字段。

5.2.4.4 其他关键性需求标准

- 可用性：系统核心功能页面加载时间不超过3秒（测试环境）；
- 数据一致性：涉及订单和支付的操作须使用事务保证数据一致性；
- 评价真实性：用户评价须在订单完成后方可提交，每笔订单限评一次。

5.2.5 计算机硬件资源利用

本项目采用以下方式分配和监控计算机硬件资源：

- 开发阶段：各成员在本地开发环境运行，配置要求为内存不低于8GB，存储空间不低于50GB；
- 部署阶段：使用云端测试服务器或本地服务器，配置为2核CPU、4GB内存、50GB存储；
- 资源监控：通过服务器系统监控工具（如top、htop）观察CPU和内存使用情况，确保系统在测试负载下资源占用合理；
- 数据库资源：合理设计索引，避免全表扫描，必要时对高频查询接口增加缓存。

5.2.6 记录原理

本项目对以下类型的决策进行记录，记录于项目Wiki或设计文档中：

- 技术选型决策：记录选择特定框架或工具的原因，包括对备选方案的比较分析；
- 架构设计决策：记录系统分层、模块划分及前后端交互方式的选择理由；
- 数据库设计决策：记录核心表结构设计的考量及权衡；
- 重要变更决策：记录需求变更、接口变更的原因及影响分析。

“关键决策”是指对系统架构、核心功能实现方式或项目计划有重大影响的决策。原理记录于系统设计说明书的相应章节及项目代码仓库的Wiki页面中。

5.2.7 需方评审途径

本项目需方为课程指导教师。评审途径如下：

- 阶段性文档评审：各阶段关键文档（需求规格说明书、设计说明书）完成后，提交指导教师审阅；
- 中期检查：在编码实现阶段中期，向指导教师展示已实现的功能原型；
- 最终验收：项目完成后进行公开演示，指导教师依据验收标准进行评审；
- 评审记录：每次评审的意见和处理结果记录于项目文档中，作为后续改进依据。

6 实施详细软件开发活动的计划

6.1 项目计划和监督

6.1.1 软件开发计划（包括对该计划的更新）

本软件开发计划在项目启动时制定，并在以下情况下进行评审和更新：每个开发阶段结束时、发生重大需求变更时、进度出现重大偏差时。计划更新须经全体项目成员确认，并记录变更原因和内容。

6.1.2 CSCI测试计划

各配置项的测试计划将在对应模块详细设计完成后制定，包括测试范围、测试用例设计方法、测试环境要求及通过准则。

6.1.3 系统测试计划

系统测试计划将在系统设计阶段完成后制定，覆盖端到端功能测试、接口集成测试及基本性能测试。

6.1.4 软件安装计划

系统部署及安装说明将在编码实现阶段结束前完成，包括环境依赖、安装步骤及配置说明，形成安装部署手册。

6.1.5 软件移交计划

不适用。本项目为课程实验项目，不涉及正式的软件移交流程。

6.1.6 跟踪和更新计划，包括评审管理的时间间隔

项目进度跟踪采用每周一次的组内进度同步会议，评审已完成工作、识别风险、更新任务状态。各阶段里程碑节点处进行正式的阶段评审。

6.2 建立软件开发环境

6.2.1 软件工程环境

- 操作系统：Windows 10/11 或 macOS（开发环境），Linux（部署环境）；
- 后端开发：IntelliJ IDEA，JDK 17，Maven；
- 前端开发：VS Code，Node.js 18.x，npm/yarn；
- 数据库工具：DBeaver 或 MySQL Workbench；
- 接口调试：Postman；
- 版本控制：Git，托管于GitHub私有仓库；
- 项目协作：GitHub Projects 或 飞书文档。

6.2.2 软件测试环境

- 单元测试框架：JUnit 5（后端）、Jest（前端）；
- 接口测试：Postman自动化测试集合；
- 测试数据库：独立的测试数据库实例，与开发库隔离；
- 测试服务器：本地或云端测试服务器，与生产部署环境配置一致。

6.2.3 软件开发库

- 代码仓库：GitHub私有仓库，采用分支策略管理（main分支为稳定版，develop分支为集成分支，feature分支为功能开发分支）；
- 依赖管理：后端通过Maven的pom.xml管理，前端通过npm的package.json管理；
- 制品库：构建产物（JAR包、前端静态文件）存储于指定目录，版本号与Git标签对应。

6.2.4 软件开发文档

项目开发文档存储于GitHub仓库的docs目录下，包括需求规格说明书、设计说明书、接口文档等，使用Markdown格式编写，与代码同步管理。

6.2.5 非交付文件

以下文件为项目内部使用，不对外交付：周会议纪要、任务分配表、内部讨论记录、测试数据集。

6.3 系统需求分析

6.3.1 用户输入分析

通过分析对标平台（美团、饿了么、大众点评）的核心用户流程，识别目标用户群体（城市居民、外卖用户、到店消费用户）的主要使用场景和操作流程，整理形成用户需求清单，作为功能需求的输入基础。

6.3.2 运行概念

系统以移动端或Web端作为用户交互入口，用户可通过平台搜索附近商家，浏览菜品/服务，完成下单支付，并在订单完成后进行评价。商家侧可管理店铺信息和处理订单。平台后台负责数据管理和推荐逻辑。

6.3.3 系统需求

系统功能性需求主要包括：

- 用户注册、登录与个人信息管理；
- 商家信息展示、分类搜索与智能推荐；
- 外卖点餐：菜品浏览、购物车管理、下单、订单状态跟踪；
- 到店团购：团购套餐浏览、下单与券码管理；
- 在线支付（模拟实现）；
- 用户评价：评分、文字评价提交与展示；
- 商家后台：商品管理、订单管理。

非功能性需求包括：系统响应时间、数据一致性、用户密码安全存储等，详见软件需求规格说明书。

6.4 系统设计

6.4.1 系统设计决策

系统采用前后端分离架构，后端提供RESTful API，前端独立部署并通过HTTP接口与后端交互。该决策的依据为：便于团队并行开发，前后端可独立测试，且符合当前主流互联网应用架构规范。

6.4.2 系统体系结构设计

系统整体分为三层：

- 表现层：前端Web应用或移动端H5，负责用户界面渲染和交互；
- 应用层：后端Spring Boot服务，包含业务逻辑处理、接口路由和数据校验；
- 数据层：MySQL关系型数据库（持久化存储）和Redis缓存（热点数据缓存）。

系统内部按业务域划分为用户模块、商家模块、订单模块、支付模块和评价模块，各模块相互独立，通过服务层接口协作。

6.5 软件需求分析

依据系统需求分析结果，针对各软件配置项开展详细的软件需求分析，输出软件需求规格说明书，内容包括：功能需求（用例描述）、非功能需求、外部接口需求、数据需求和约束条件。需求文档须经全体成员评审确认后基线化。

6.6 软件设计

6.6.1 CSCI级设计决策

各主要配置项的设计决策包括：

- 用户认证模块采用JWT无状态鉴权方案；
- 商家推荐采用基于距离和评分的简单综合排序算法；
- 订单状态机设计为：待支付→已支付→配送中→已完成/已取消；
- 支付模块采用模拟支付接口，预留真实支付接口扩展点。

6.6.2 系统体系结构设计

详见6.4.2条。各配置项间通过Spring Bean依赖注入管理，服务间通过定义清晰的Service接口进行交互，禁止跨层直接调用。

6.6.3 CSCI详细设计

详细设计文档包括：数据库表结构设计（含字段定义、索引设计、ER图）、核心类设计（类图、方法签名）、接口详细规范（请求/响应格式、错误码定义）。详细设计说明书在设计阶段结束时提交。

6.7 软件实现和配置项测试

6.7.1 软件实现

按照详细设计文档分模块并行编码，遵循5.2.2条规定的编码标准。代码提交前须在本地通过单元测试，并经过至少一名成员代码审查后合并至develop分支。

6.7.2 配置项测试准备

各模块编码完成后，编写对应的单元测试用例，覆盖正常流程、边界条件和异常处理，搭建独立测试环境，准备测试数据。

6.7.3 配置项测试执行

在测试环境中执行单元测试和模块级接口测试，记录测试结果。

6.7.4 修改和再测试

对测试中发现的缺陷，在GitHub Issues中登记，由责任人修复后执行回归测试，确认缺陷已消除且未引入新问题。

6.7.5 配置项测试结果分析与记录

汇总各模块测试结果，记录测试通过率、发现缺陷数量及修复情况，形成模块测试报告。

6.8 配置项集成和测试

6.8.1 配置项集成和测试准备

各模块单元测试通过后，制定集成测试计划，定义集成顺序（建议按照用户模块→商家模块→订单模块→支付模块→评价模块的顺序集成），准备集成测试用例和测试环境。

6.8.2 配置项集成和测试执行

按集成顺序逐步将各模块合并至集成分支，执行接口联调测试，验证模块间交互的正确性。

6.8.3 修改和再测试

对集成测试中发现的接口兼容性问题和交互缺陷进行修复，修复后执行回归测试。

6.8.4 配置项集成和测试结果分析与记录

记录集成测试过程中发现的问题、修复情况及最终集成测试结论，形成集成测试报告。

6.9 CSCI合格性测试

6.9.1 CSCI合格性测试的独立性

由未直接参与对应模块开发的项目成员承担合格性测试工作，以保证测试的客观性。

6.9.2 在目标计算机系统（或模拟的环境）上测试

合格性测试在模拟生产环境的测试服务器上进行，环境配置与最终部署环境保持一致。

6.9.3 CSCI合格性测试准备

依据软件需求规格说明书中的功能需求，设计合格性测试用例，覆盖所有主要功能点，准备测试数据和测试脚本。

6.9.4 CSCI合格性测试演练

在正式执行前进行一次测试演练，验证测试环境和测试数据的有效性，修正测试流程中的问题。

6.9.5 CSCI合格性测试执行

依据测试用例逐项执行功能验证，记录每个测试用例的执行结果（通过/失败）。

6.9.6 修改和再测试

对未通过的测试用例，定位问题并修复，修复后执行针对性回归测试。

6.9.7 CSCI合格性测试结果分析与记录

统计测试通过率，分析未通过原因，形成合格性测试报告，确认是否达到验收标准。

6.10 CSCI/HWCI集成和测试

不适用。本项目为纯软件系统，不涉及硬件配置项（HWCI）。

6.11 系统合格性测试

6.11.1 系统合格性测试的独立性

系统合格性测试由项目组全员参与，测试用例执行由非功能开发者担任主测角色。

6.11.2 在目标计算机系统（或模拟的环境）上测试

系统合格性测试在最终部署环境（云端或本地服务器）上执行，确保测试结果反映真实运行情况。

6.11.3 系统合格性测试准备

依据验收标准（见3.5条）制定系统级测试方案，包括端到端业务流程测试用例和基本性能测试方案。

6.11.4 系统合格性测试演练

进行一次完整的系统演练，模拟用户从注册到下单完成的全流程，验证测试流程的可行性。

6.11.5 系统合格性测试执行

执行全部系统测试用例，覆盖所有主要业务场景，记录测试结果。

6.11.6 修改和再测试

对系统级测试中发现的问题进行修复，修复后执行回归测试，确认问题已解决且不影响其他功能。

6.11.7 系统合格性测试结果分析与记录

汇总系统测试结果，形成系统测试报告，确认系统是否满足验收标准，作为项目交付的依据。

6.12 软件使用准备

6.12.1 可执行软件的准备

完成后端服务的JAR包构建和前端静态资源的构建，确保构建产物可在目标环境正常运行。

6.12.2 用户现场的版本说明准备

编写版本说明文档，记录当前版本实现的功能范围、已知问题及注意事项。

6.12.3 用户手册的准备

编写用户手册，说明平台的主要功能使用方法，包括用户注册、商家搜索、下单支付、评价提交等核心操作流程，配以界面截图。

6.12.4 在用户现场安装

完成系统在测试服务器或演示环境的部署，按安装部署手册执行安装，验证系统在目标环境正常运行后，通知指导教师进行验收。

6.13 软件移交准备

不适用。本项目为课程实验项目，不涉及正式的软件移交至支持机构的流程。

6.14 软件配置管理

6.14.1 配置标识

所有配置项（源代码、文档、构建脚本、配置文件）统一纳入Git版本控制管理，以Git提交哈希和标签（Tag）作为配置标识。主要里程碑版本打Tag标记，格式为 `v{主版本}.{次版本}.{修订号}`，如 `v1.0.0`。

6.14.2 配置控制

代码变更通过Pull Request（PR）流程进行，PR须经至少一名其他成员审查通过后方可合并。禁止直接向main分支推送代码。文档变更须在文档内记录变更日期和内容。

6.14.3 配置状态统计

通过GitHub的提交历史、PR记录和Issues跟踪系统，随时查询各配置项的当前版本、变更历史和问题状态。

6.14.4 配置审核

在各阶段里程碑处，对代码库和文档库进行一次配置审核，确认实际提交内容与计划一致，版本标记正确，无遗漏文件。

6.14.5 发行管理和交付

项目最终交付时，在GitHub上发布正式Release，附带完整的可执行程序、源代码和文档压缩包，作为课程大作业的最终提交版本。

6.15 软件产品评估

6.15.1 中间阶段的和最终的软件产品评估

在需求分析、设计、编码各阶段结束时，项目组对阶段产出物进行内部评估，检查其完整性、一致性和质量；最终交付前进行全面的软件产品评估，对照验收标准逐项确认。

6.15.2 软件产品评估记录

各阶段评估结果记录于项目Wiki中，包括评估日期、参与人员、评估结论和问题清单。

6.15.3 软件产品评估的独立性

各阶段文档和代码由未直接编写该部分内容的成员参与评估，以提高评估客观性。

6.16 软件质量保证

6.16.1 软件质量保证评估

通过代码审查、单元测试覆盖率检查、静态代码分析（如CheckStyle、ESLint）等手段，对软件质量进行持续保证。

6.16.2 软件质量保证记录，包括所记录的具体条目

质量保证记录内容包括：代码审查意见及处理结果、测试覆盖率统计、静态分析警告处理情况、缺陷统计与修复率。

6.16.3 软件质量保证的独立性

代码审查由非代码作者执行，测试由非对应模块开发者参与，以保证质量保证活动的独立性。

6.17 问题解决过程（更正活动）

6.17.1 问题/变更报告

使用GitHub Issues管理问题和变更请求，每个Issue记录以下信息：项目名称、提出者、问题编号（GitHub自动生成）、问题名称、受影响的软件模块或文档、发现日期、问题类别（缺陷/变更请求/疑问）、优先级（高/中/低）、问题描述、指派的负责人、指派日期、预计完成日期、推荐的解决方案、影响范围、问题状态（待处理/处理中/已完成/关闭）。

6.17.2 更正活动系统

问题负责人在收到指派后，在GitHub Issue中更新处理进展，代码修复通过Pull Request提交，PR中关联对应Issue编号。修复合并后Issue状态更新为已完成，并由提出者确认关闭。

6.18 联合评审（联合技术评审和联合管理评审）

6.18.1 联合技术评审包括一组建议的评审

建议开展以下联合技术评审：

- 需求评审：需求规格说明书完成后，全员评审，确认需求完整性和一致性；
- 设计评审：概要设计和详细设计完成后，全员评审，确认设计合理性；
- 代码走查：各模块编码完成后，进行代码走查，重点关注核心业务逻辑和安全处理；
- 测试用例评审：测试用例设计完成后，评审其覆盖完整性。

6.18.2 联合管理评审包括一组建议的评审

建议开展以下联合管理评审：

- 项目启动评审：确认计划可行性、任务分工和进度安排；
- 中期进度评审：在编码阶段中期，评审进度执行情况和风险状态；
- 交付前评审：确认所有交付物准备就绪，符合验收标准。

6.19 文档编制

文档编制采用Markdown格式，存储于代码仓库的docs目录下，与代码同步管理。各类文档按照模板要求编写，由负责人起草后，经全员评审修改后定稿。文档须与实际实现保持一致，接口文档使用Swagger自动生成并辅以手工说明。文档版本与代码版本对应，重要版本同步打Tag。

6.20 其他软件开发活动

6.20.1 风险管理，包括已知风险和相应的对策

风险编号	风险描述	发生概率	影响程度	对策
R01	需求理解不一致，导致返工	中	高	需求文档评审确认，及早冻结需求基线
R02	技术难点（如支付集成、推荐算法）耗时超预期	中	中	提前技术预研，采用模拟实现降低复杂度
R03	团队成员进度不协调，集成困难	中	高	明确接口规范，定期同步进度，尽早开始联调
R04	开发环境不统一导致兼容性问题	低	中	统一技术栈版本，使用Docker容器化开发环境
R05	测试时间不足，存在遗漏缺陷	中	中	开发与测试并行，预留足够的测试和修复时间

6.20.2 软件管理指标，包括要使用的指标

- 需求完成率：已实现需求数/总需求数；
- 缺陷密度：发现缺陷数/代码规模（KLOC）；
- 缺陷修复率：已修复缺陷数/发现缺陷总数；
- 进度偏差率：实际完成工作量与计划工作量的偏差百分比；
- 单元测试覆盖率：覆盖的代码行数/总代码行数。

6.20.3 保密性和私密性

详见5.2.4.2条和5.2.4.3条。

6.20.4 承包方管理

不适用。本项目无外部承包方。

6.20.5 与软件独立验证与确认（IV&V）机构的接口

不适用。本项目不设置独立的验证与确认机构。

6.20.6 和有关开发方的协调

项目组成员间通过即时通讯工具（微信群）和GitHub进行日常协调，每周举行一次线上或线下进度同步会议，确认各模块开发状态、接口联调计划和问题解决情况。

6.20.7 项目过程的改进

项目结束后，开展项目复盘，总结开发过程中的经验教训，包括哪些实践有效、哪些环节存在改进空间，形成项目总结报告，供后续项目参考。

6.20.8 计划中未提及的其他活动

无。

7 进度表和活动网络图

7.1 项目总体进度表

阶段	主要活动	里程碑	计划完成时间
需求分析	需求调研、用例分析、编写需求规格说明书、需求评审	需求规格说明书基线化	第7-8周
系统设计	系统架构设计、数据库设计、接口设计、编写设计说明书、设计评审	设计说明书基线化	第9-10周
编码实现	环境搭建、各模块并行开发、代码审查、单元测试	所有模块编码完成，单元测试通过	第11-12周
集成与测试	模块集成、集成测试、系统功能测试、缺陷修复	系统测试通过，达到验收标准	第13-14周
部署与交付	系统部署、文档整理、验收准备、演示	全部交付物提交，通过验收	第15周

7.2 活动网络图（关键路径）

项目活动依赖关系如下：

需求分析完成 → 系统设计开始；系统设计完成 → 各模块并行编码开始；所有模块编码完成 → 集成测试开始；集成测试通过 → 系统功能测试开始；系统功能测试通过 → 部署与交付。

关键路径为：需求分析→系统设计→编码实现→集成测试→系统测试→部署交付，任何关键路径上的活动延误都将直接影响项目最终交付时间。

8 项目组织和资源

8.1 项目组织

本项目采用扁平化小组结构，设项目负责人1名，其余成员各负责一个或多个功能模块的开发与对应文档编写。各成员在完成主要职责的同时，参与其他模块的代码审查和测试工作。

项目组织结构如下：

- 项目负责人：孙伟宸，负责项目整体协调、进度跟踪、文档统筹及系统架构决策；
- 后端开发负责人：黄羿，负责后端服务框架搭建、用户模块与商家模块开发；
- 后端开发：孙铭，负责订单模块与支付模块开发；
- 前端开发负责人：钟其宏，负责前端框架搭建、主要页面开发与UI设计；
- 前端开发及测试负责人：戴昊，负责前端功能页面开发、系统测试及测试文档编写。

各成员同时参与需求分析讨论和系统设计评审，确保对整体系统有充分理解。

8.2 项目资源

人力资源：

项目投入总人力估算约为25人周（5人×5周有效开发工作量）。

成员	职责	技术级别
孙伟宸	项目负责人、架构设计、后端开发	高年级本科生，具备全栈开发基础
黄羿	后端开发（用户、商家模块）	高年级本科生，具备Java后端开发经验
孙铭	后端开发（订单、支付模块）	高年级本科生，具备Java后端开发经验
钟其宏	前端开发、UI设计	高年级本科生，具备前端开发经验
戴昊	前端开发、测试	高年级本科生，具备前端开发及测试基础

开发设施：

各成员使用个人计算机（内存不低于8GB）作为开发工作站，系统部署使用云端测试服务器或实验室服务器。

所需软硬件资源：

资源类型	具体内容	需要时间
云端测试服务器	2核CPU、4GB内存、50GB存储	编码阶段开始时
MySQL数据库	8.x版本，部署于服务器	编码阶段开始时
Redis缓存服务	7.x版本，部署于服务器	编码阶段开始时
GitHub私有仓库	代码托管及协作	项目启动时

9 培训

9.1 项目的技术要求

本项目涉及的主要技术包括：后端Spring Boot框架及相关生态（MyBatis-Plus、Spring Security、JWT）、前端Vue.js或React框架及组件库、MySQL数据库设计与优化、Redis缓存使用、Git版本控制与协作流程、RESTful API设计规范、基本的软件测试方法。

9.2 培训计划

经评估，项目组成员已具备上述大部分技术的基础知识，可胜任各自负责模块的开发工作。对于成员尚不熟悉的技术点（如Spring Security与JWT集成、Redis缓存策略），采用自学与组内技术分享相结合的方式在项目启动前两周内完成技术准备，不单独安排系统性培训课程。

10 项目估算

10.1 估算规模

本项目软件规模估算采用功能点方法进行粗略估算，主要功能模块包括：

模块	主要功能	估算代码量
用户模块	注册、登录、个人信息管理	约800行
商家模块	商家信息展示、搜索、推荐	约1200行
订单模块	购物车、下单、订单管理	约1500行
支付模块	模拟支付、支付状态管理	约600行
评价模块	评价提交、展示	约600行
前端页面	各功能页面实现	约5000行
合计	—	约9700行

10.2 工作量估算

阶段	估算工作量（人天）
需求分析	10
系统设计	10
编码实现	30
测试与缺陷修复	15
部署与文档整理	5
合计	70

10.3 成本估算

本项目为课程实验项目，主要成本为团队人力成本（不计货币成本）和云服务资源费用。云端测试服务器预计费用约为100-200元人民币（按月计费），在合理范围内由项目组成员分摊。

10.4 关键计算机资源估算

资源	规格	用途
测试服务器CPU	2核	后端服务运行
测试服务器内存	4GB	应用程序及数据库
测试服务器存储	50GB	应用部署、数据库存储、日志
数据库存储	10GB	业务数据（含测试数据）

10.5 管理预留

考虑到需求变更、技术难点及团队协作等不确定因素，在工作量估算基础上保留约15%的管理预留（约14人天），用于应对计划外工作。

11 风险管理

本章对项目可能存在的主要风险进行分析，并制定相应的对策和管理计划。

风险编号	风险类型	风险描述	发生概率	风险等级	预防对策	应急对策
R01	需求风险	需求理解不一致，后期发生较大范围变更	中	高	尽早开展需求评审，及时冻结需求基线	评估变更影响，调整后续计划，必要时裁剪低优先级功能
R02	技术风险	关键技术（如分布式事务、推荐算法）实现难度超出预期	中	中	项目启动前开展技术预研，选择成熟方案，降低技术复杂度	采用简化替代方案（如本地事务、简单排序算法）保证核心功能可用
R03	进度风险	团队成员进度不一致，模块集成延误	中	高	提前定义清晰的接口规范，定期进度同步，及时发现滞后	优先保证核心功能集成，次要功能延后实现
R04	质量风险	测试时间不足，存在较多遗留缺陷影响验收	中	中	开发与测试并行，每个模块完成后立即进行单元测试	优先修复阻断性缺陷，非核心缺陷在验收时说明并记录
R05	环境风险	开发环境不统一导致兼容性问题	低	低	项目启动时统一约定技术栈版本，使用配置文件管理依赖	及时在组内同步环境配置，协助成员解决环境问题

12 支持条件

12.1 计算机系统支持

本项目所需的计算机系统支持包括：各成员个人开发计算机（自备）、云端测试服务器（项目组自行申请，用于系统部署和集成测试）、GitHub代码托管平台（免费版本满足需求）。

12.2 需要需方承担的工作和提供的条件

需方（课程指导教师）需提供的条件包括：

- 明确的课程大作业需求说明和验收标准；
- 阶段性文档评审的反馈意见；
- 最终答辩或演示的时间安排。

12.3 需要承包方承担的工作和提供的条件

不适用。本项目无外部承包方参与。

13 注解

13.1 术语说明

- SDP（Software Development Plan）：软件开发计划书；
- CSCI（Computer Software Configuration Item）：计算机软件配置项，即软件系统中可独立配置管理的软件组成单元；
- HWCI（Hardware Configuration Item）：硬件配置项；
- RESTful API：一种基于HTTP协议的软件架构风格，用于设计网络应用程序接口；
- JWT（JSON Web Token）：一种用于身份认证的开放标准令牌格式；
- ORM（Object-Relational Mapping）：对象关系映射，用于在面向对象语言和关系型数据库之间进行数据转换的技术；
- IV&V（Independent Verification and Validation）：独立验证与确认；
- PR（Pull Request）：代码合并请求，Git协作流程中的代码审查机制；
- MVC（Model-View-Controller）：一种软件设计模式，将应用程序分为模型、视图和控制器三层。

13.2 缩略语表

缩略语	含义
SDP	Software Development Plan，软件开发计划
CSCI	Computer Software Configuration Item，计算机软件配置项
HWCI	Hardware Configuration Item，硬件配置项
API	Application Programming Interface，应用程序接口
JWT	JSON Web Token
ORM	Object-Relational Mapping，对象关系映射
MVC	Model-View-Controller，模型-视图-控制器
IV&V	Independent Verification and Validation，独立验证与确认
PR	Pull Request，代码合并请求
ER	Entity-Relationship，实体关系
KLOC	Kilo Lines of Code，千行代码

附录

附录A 参考对标平台功能对比

功能模块	美团	饿了么	大众点评	本平台
外卖点餐	支持	支持	不支持	支持
到店团购	支持	支持	支持	支持
商家搜索推荐	支持	支持	支持	支持
用户评价	支持	支持	支持	支持
在线支付	支持	支持	支持	模拟支持
配送跟踪	支持	支持	不支持	状态展示
休闲娱乐服务	支持	不支持	支持	基础支持

附录B 项目成员分工明细

成员	主要职责	负责文档
孙伟宸	项目负责人、系统架构设计、后端评价模块开发、代码审查	软件开发计划书、系统设计说明书（架构部分）
黄羿	后端框架搭建、用户模块开发、商家模块开发	系统设计说明书（后端设计部分）、接口文档
孙铭	订单模块开发、支付模块开发、数据库设计	数据库设计说明、需求规格说明书（订单需求部分）
钟其宏	前端框架搭建、主要功能页面开发、UI交互设计	用户手册、前端组件设计文档
戴昊	前端功能页面开发、系统测试设计与执行、测试报告编写	测试报告、安装部署手册